CAFÉ DE LA LUNA · RESEARCH SERIES

Evgeniy Tyurkin

18 September 2026

Fishing the Stream

Field Notes on Reconstructing ChatGPT Architecture from Client Behaviour



0–10 · Author's Edition
Not for Sale

Eternal Cycle Press
Krasnoyarsk · 1979

CAFÉ DE LA LUNA · RESEARCH SERIES

Fishing the Stream

Field Notes on Reconstructing ChatGPT Architecture from Client Behaviour

Evgeniy Tyurkin

18 September 2026

Fishing the Stream

Field Notes on Reconstructing ChatGPT Architecture from Client Behaviour

Evgeniy Tyurkin
18 September 2026

> **Evidence is available upon request.**

CAFÉ DE LA LUNA · RESEARCH SERIES
0–10 · Author's Edition
Not for Sale

Eternal Cycle Press
Krasnoyarsk · 1979

© 2026 Evgeniy Tyurkin
All rights reserved.

I spent part of the last few weeks doing something that felt less like reverse engineering and more like fishing.

Not network sniffing. Not protocol capture. Not privileged telemetry.

Just watching the client very carefully.

A status message would appear for a second:

```
Compiling song interpretations
```

Then another:

```
Streaming interrupted.
Waiting for the complete
message...
```

Sometimes the fish was large enough to photograph.

Sometimes it flashed once and disappeared before my fingers reached the screenshot buttons.

Screen recording became a net.

The client became a river.

And the job was simply to sit there long enough to notice what floated past.

That is the method behind these field notes.

I had no internal access to OpenAI systems, no internal

documentation, and no ability to inspect production services. Everything here comes from visible product behaviour, screenshots, repeated experiments, and one rule:

> **Observation is not architecture.**

A reproducible symptom can justify a hypothesis.

It does not magically reveal the implementation.

That distinction matters, because black-box systems are very good at inviting elegant explanations that are not actually evidence.

Still, if you watch long enough, some patterns stop looking like coincidences.

And a few of them become bugs.

The observations below are the catch from this investigation, not a promise to keep fishing indefinitely.

The narrative follows the investigation in roughly chronological order: artifacts and context first, then routing and tool behaviour, then failure and recovery, and finally the limits exposed by a long-lived conversation. Non-English UI strings and user messages are translated into English in the

prose and tables; the screenshots preserve the original wording.

1. It Started With a Path

One of the first clues was almost embarrassingly small.

A generated image surfaced with a local-looking path:

```
/mnt/data/generated_image.png
```

Later, similar /mnt/data/... paths appeared again.

The interesting part was not that Unix-like paths exist.

The interesting part was that the path leaked into the user-visible layer at all.

Observed

- /mnt/data/... sometimes appeared directly in the UI.
- The path could remain visible after the underlying image was no longer accessible.
- Knowing the path did not imply access to the bytes behind it.
- The same path could later exist only as text.

That immediately forced a distinction I had been lazily ignoring:

```
description != filename !=
path != metadata != bytes
```

A system can remember that an artifact existed without still possessing the artifact.

Working hypothesis

The path belongs to some temporary execution environment, mounted workspace, sandbox, or task-local filesystem.

I do **not** know whether that means Docker, a VM, a per-chat container, a per-task runtime, or something more complicated.

The safe statement is much smaller:

> **Knowledge of the file and access to the file have different lifetimes.**

In my field-note shorthand:

> **The path is known. The whereabouts are not.**

2. Temporary Files and Conversation State Age Differently

Generated artifacts sometimes disappeared while their textual traces survived.

In a few observations, the lifetime looked roughly like part of a day. That is nowhere near

enough evidence to claim a fixed TTL.

What mattered was the separation.

Observed

- a conversation could still refer to an old image after the image itself was inaccessible;
- a later turn could remember what the artifact represented;
- the artifact's bytes did not necessarily survive alongside the conversational description.

Working hypothesis

Temporary artifacts are probably subject to cleanup, garbage collection, TTL expiration, workspace teardown, or some combination.

The textual or semantic representation survives longer because it belongs to a different subsystem.

This led to one of my favourite names for old context:

The Refrigerator

Not "cold storage."

That sounds too much like a real internal component name.

Refrigerator is clearly a metaphor.

And if something stays there too long, it enters:

> **Museum of Expired Food**
> *Some content may still be editable.*

This is not just a joke.

It captures a useful behaviour: old context may stop being active and still remain retrievable, summarizable, or reusable later.

Expired does not mean useless.

2.1 Paste Is Not Always Text

One more clue surfaced while working with a pasted web archive.

On iOS, rich pasted web content did not behave like a simple block of inline text. The client briefly exposed a sequence of status messages:

```
Inspecting webarchive file structure
```

then:

```
Parsed webarchive metadata and extracted plain text content
```

and later:

```
Searching the Latest Uploaded File
```

The wording is unusually suggestive.

A naive mental model for paste would be:

```
clipboard text
-> prompt
```

The visible behaviour is more consistent with something like:

```
rich clipboard/web content
-> .webarchive artifact
-> upload/file reference
-> metadata + text
extraction
-> resolve the uploaded
artifact
-> continue the task using
extracted content
```

That does **not** prove the exact implementation.

The status labels may be human-readable summaries generated for the UI rather than literal internal function names.

But the behavioural conclusion is useful:

> **Paste is not always text. Sometimes it becomes a file first.**

This also explains a class of otherwise strange UI behaviour: a user may think they pasted content directly into a conversation, while the product may first materialize that content as a file-like artifact and pass it through the same ingestion machinery used for uploads.

The particularly interesting phrase is:

```
Searching the Latest Uploaded File
```

because it suggests a separation between:

- creating or uploading the artifact;
- parsing it;
- and resolving which artifact the current task should consume.

That is another small example of the larger theme of this investigation:

what looks like one user action can fan out into several

asynchronous operations with different lifetimes and failure modes.

3. Chat History Is Not Active Context

Another early conceptual mistake was treating "the conversation" as if it were identical to "what the model sees right now."

The observed behaviour strongly suggests otherwise.

A more useful mental model is to separate:

```
full conversation history
active context for this
turn
```

```
persistent user-level
memory
retrieved older details
temporary tool state
temporary file state
```

These may interact.

They are not the same thing.

Observed

The assistant can sometimes recover surprisingly specific details from earlier work without behaving as if every historical token is permanently loaded into the active prompt.

Working hypothesis

The active turn is assembled from selected material, possibly including:

- a compact summary;
- persisted user facts;
- current conversation context;
- retrieved older details;
- tool outputs.

The exact mechanism is unknown.

I sometimes describe old context as sitting "in the refrigerator" until needed.

That is deliberately non-technical language.

It prevents a useful metaphor from turning into a fake claim about internal architecture.

4. The Schrödinger Turn

Then the UI started getting interesting.

I observed cases where the useful work appeared to be complete, yet the client still behaved as if the turn were active.

Sometimes the composer remained blocked.

Sometimes a spinner kept going.

Sometimes a reload reproduced the same state.

The turn was both finished and not finished.

Observed

- visible activity could continue after meaningful output was done;
- the client could remain blocked for a long time;
- reloading did not necessarily repair the state;
- the Stop control could be visible but ineffective.

A useful behavioural split became:

```
execution/job state
conversation turn state
client UI state
```

A failure could plausibly look like:

```
job = DONE
turn = ACTIVE
ui = WAITING
```

No internal access is required to see that these concepts are behaviourally separable.

> **Work completed. Waiting for completion continues.**

5. The Stop Button That Could Not Stop

The non-working Stop button deserved more than a footnote.

The obvious mental model for cancellation is:

```
STOP NOW
```

But the observed behaviour suggested a darker possibility:

```
enqueue(cancel)
...
eventually process(cancel)
```

Working hypothesis

Cancellation may travel through the same coordinator, queue,

state machine, or delivery mechanism as the work it is trying to cancel.

If that path is already stuck, delayed, or waiting on state reconciliation, the cancellation request itself may wait behind the problem.

That would turn:

STOP NOW

into:

STOP WHEN CONVENIENT

This is not proven architecture.

But it is a plausible explanation for a visible Stop control that does not immediately correspond to an effective cancellation path.

The deeper lesson is broader:

> *A UI can know that something is "active" without possessing a currently valid way to stop whatever owns that activity.*

6. Connection-Oriented Delivery Is Not Reliable Delivery

This part felt familiar long before ChatGPT.

Long-lived streaming connections fail for ordinary reasons: mobile handoff, backgrounding, timeouts, proxies, restarts, and

transient server interruptions. The important design question is not whether the connection stays open forever. It is what the system can recover when it does not.

For chat, a missed event may leave a spinner running until the client reconciles. In systems where an event means "transaction committed" or "workflow finished," the same assumption becomes much more serious.

The old principle is simple:

> **Connection-oriented delivery is not the same thing as reliable delivery.**

Reliable delivery needs durable state and an explicit recovery story: acknowledgement, replay, re-fetch, idempotency, sequence tracking, or reconciliation.

That distinction becomes concrete later, when the client explicitly reports that streaming was interrupted and then continues waiting for completion anyway.

7. Thinking Can Outlive the Thinking

In one case, the UI kept displaying **Thinking** after the useful work had effectively finished.

The work is finished. The indicator was not informed.

That matters because activity indicators look like telemetry but can behave like eventually consistent projections. The thing on screen may be a delayed interpretation of state, not the state itself.

8. The Image That Successfully Failed

One of the cleaner clues came from image generation.

The sequence was:

1. an image was generated successfully;
2. it appeared in the UI;
3. it was saved;
4. a new user message was sent;
5. the UI briefly displayed a status similar to:

```
Generating warm reply
```

6. then the interface reported:

```
Thinking

Something went wrong while
generating your image.
Sorry about that.
```

The image already existed.

The next turn had apparently moved into a text-response phase.

Then an image-generation error surfaced afterward.

Possible explanations

- a late error event;
- a race condition;
- stale task state;
- wrong turn/task correlation;
- a client state machine applying an old event to a newer turn.

I cannot prove which.

But I can say this:

> **The image was saved. Image generation failed.**

That contradiction was visible.

And it suggested that image-task state, conversation state, and client-rendered state are not one indivisible thing.

9. Status Labels That Sound Suspiciously Human

During different tasks, the UI showed labels like:

```
Designing a moonlit diagram
Preparing topography illustration
Inspecting keyboard completion
Reviewing diagram visuals
```

```
Preparing evidence package
Shaping the conclusion
```

Other labels appeared in another language while the surrounding interface remained unchanged.

More interestingly, the language could change **inside the same task**.

During one interaction, the status first appeared as:

```
Clarifying a joke’s origin
```

and then, moments later, as a Russian-language status that translates to:

```
Catching the joke
```

The underlying task had not changed. The visible narration had.

That is stronger evidence that these labels are not literal backend state names. They behave more like dynamically generated, human-readable summaries of the current operation, with wording and localization that can be recomputed while the task is still running.

It also offers a plausible explanation for earlier one-off language drift: the odd foreign-language fragment may belong to the same family of status-generation behaviour rather than to the core task itself.

These did not look like low-level enum names.

They looked like miniature semantic summaries of what the system believed it was doing.

Working hypothesis

At least some thinking/status labels are dynamically generated from the meaning of the current task.

Therefore:

```
status label != guaranteed backend state name
```

This is a small point, but it matters when reverse-engineering from screenshots.

The UI may be narrating activity, not exposing activity.

9.1 Generated-Looking Labels Do Not Identify Workers

Later screenshots made simple UI localization an incomplete explanation. Translated into English where needed, the visible narration changed language and specificity:

```
Analyzing statuses
Classifying run states
Thinking
```

The surrounding conversation was mostly Russian, yet an unexpected Dutch word had also appeared inside an otherwise

Russian generated title. That makes ordinary user code-switching a poor explanation for every language change.

One possible model is that an execution component returns a machine status plus a short human-readable summary, which the client then displays. A different path could map failures to fixed English descriptions such as:

```
Stopped thinking
Stopped analyzing
Error analyzing
The request timed out.
Streaming interrupted...
```

That model would explain the contrast between contextual progress narration and dry failure wording. But it is only one model. A single multilingual generator, task-specific prompts, tool-result captions, or client-side presentation could produce the same visible behaviour.

The evidence therefore supports a narrower statement:

> **The interface exposes distinguishable activity and failure labels. Their mapping to internal task classes remains unknown.**

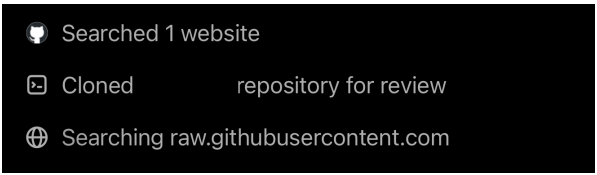
9.2 One Status Area, Several Kinds of Information

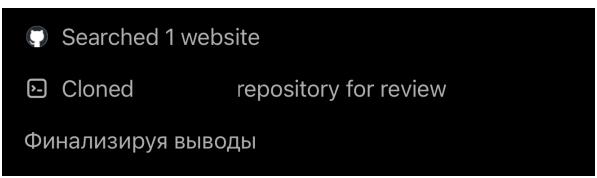
The evening screenshots made a more precise distinction possible. Rather than treating every changing line as a worker announcement, I compared what changed with what stayed on screen.

The frames below are listed in the supplied order; they are screenshots, not a recovered event log.

The repository review: history rows stay, activity changes

Frame Rows that remain visible Changing lower line
S01 Searched 1 website; Cloned [project] repository for review Searching raw.githubusercontent.com
S02 The same two rows Searching the web
S03 The same two rows Finalizing conclusions





The first two rows are phrased as completed actions. The lower line changes from a particular domain to a generic search description and then to a task summary, "Finalizing conclusions."

That is observable separation between **retained activity entries** and **a changing activity caption**. It does not establish that a clone actually succeeded, that a new worker took over, or that

one job completed before another was queued. Cloned … is what the interface reported; the screenshots contain no filesystem verification or clone exit status.

Likewise, the transition from a domain name to Searching the web does not itself show a failed request or a retry. It may be a change in how activity is presented.

A search exchange: count, query, summary, icon

A second sequence shows another set of changes:

Frame

Visible wording

Leading icon

S04

Searching 14 websites

A small multicolored icon

S05

Searching "<artist>" "<song>" "Lyrics" "<lyric fragment>"

A globe

S06

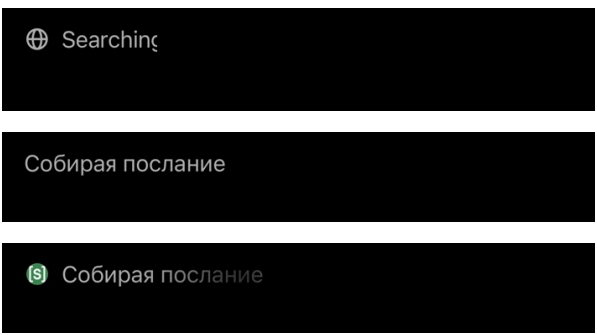
Putting the message together

No icon visible

S07

Putting the message together

A green icon



Here the displayed information changes category: a website count, a query string, and a contextual summary meaning "Putting the message together."

Then the summary remains the same while an icon appears beside it.

This is a useful correction to my earlier reading. A changing icon is not a worker ID. It might be source branding, tool decoration, or a delayed presentation update. I cannot identify a service from its colour. 14 websites is a displayed count, not evidence of fourteen parallel workers, fourteen successful fetches, or fourteen model calls.

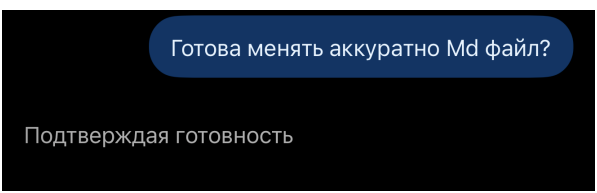
The narrower finding is still useful:

> **Status wording and its visual decoration can change separately. A search count, a query, and a task summary should not automatically be read as equivalent lifecycle events.**

Even a readiness question gets a task-shaped caption

A readiness question about editing the Markdown file is followed by a caption translated here as:

```
Confirming readiness
```



The caption follows the meaning of the question. It does not tell us that there is a dedicated readiness worker. It also does not establish when file reading, copying, or other work began or ended.

Taken together, these frames suggest a status area containing several presentation roles: activity history, task narration, search detail, counts, and icons. They remain compatible with worker-generated summaries forwarded verbatim. They also remain compatible with a single coordinator rendering different kinds of updates.

> **Before assigning a box in the architecture, identify what kind of thing the screen is showing.**

9.3 The UI Reported a Repository Clone

Looking again at S01-S03, one detail deserves its own treatment. The important clue is not merely the GitHub logo. It is the verb:

```
Cloned [project] repository
for review
```

The interface did not just say it was reading about a project. It reported a concrete operation on a named repository. I had supplied a repository link and asked the assistant to look at the implementation; I had not explicitly requested a clone.

In S01, the visible items are:

```
[GitHub icon]    Searched 1
website
[terminal icon] Cloned
[project] repository for
```

```
review
[globe icon]    Searching
raw.githubusercontent.com
```

The clone caption remains visible in S02 and S03 while the lower activity description changes. As noted above, that is a retained report of an action, not a command log or an exit status.

I had also noticed source branding such as Spotify around search and source displays. Even when the logo is recognizable, it does not identify the retrieval mechanism. A source logo is not evidence of a dedicated Spotify tool, a Spotify API call, or a separate worker. Likewise, a GitHub logo alone would not establish a clone. The explicit

operation caption is the more informative clue.

A tool-selection hypothesis, not another worker count

If the clone caption accurately describes what happened, the agent chose to obtain repository content for direct analysis rather than relying only on web search results. A plausible workflow would be:

```
repository reference
    -> obtain a local copy
    -> inspect directories
and search
definitions/references
    -> read selected code
```

```
    -> return findings to
the reasoning step
```

In this design, acquiring a repository need not mean loading every file into the model's context. Local tools could inspect the material and return selected fragments. Nor would it require a special repository-cloning model: a command-execution tool with Git and suitable access would suffice.

This could be an optimization for a substantial code review: retrieve material once, then search close to the data. For one README, a whole clone could instead be unnecessary overhead. I have no timing comparison

here, so improved efficiency remains a proposed benefit, not a measured result.

There is also an important limit to the word *instead*. The screenshots still show web search alongside the reported clone. The evidence fits a combination of methods, not the elimination of search. The later domain search might have supplemented local inspection or been a fallback; these frames do not distinguish the cases.

This also qualifies the earlier worker-specialization hypothesis. Different reported operations can be explained by one agent selecting different tools. They do

not require separate reasoning workers for each action.

> **The UI reported repository cloning alongside web search. That is consistent with task-appropriate tool selection, but the screenshot alone does not verify a successful clone, local analysis, or a performance gain.**

The useful distinction is between **where material comes from**, **which operation obtains it**, and **which component reasons about it**. A logo, a tool caption, and a worker hypothesis answer different questions.

9.4 Keep the Follow-up in the Timeline

These captures add a small correction, not another architecture. Source icons and the reported repository clone are already covered in Section 9.3. The new detail is where the next user message sits relative to a stopped label and subsequent activity.

Editing captions, a stopped label, then Hello?

One capture displayed these three lines [S10]:

```
Updated Fishing the Stream
analysis
[terminal-like icon]
```

```
Inspected and updated
Fishing the Stream v9
Packaging the revision
```

These are reports of work and a packaging caption, not independent verification of file contents.

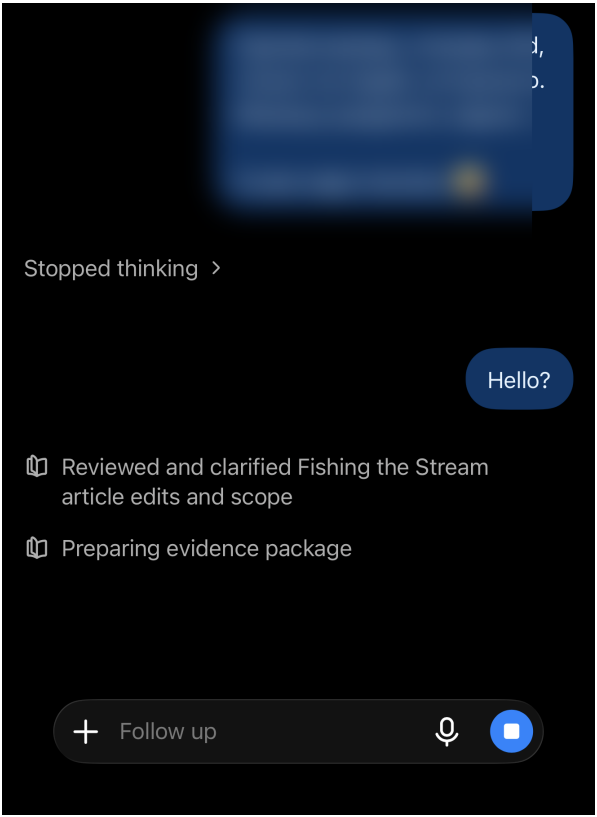
In the next supplied frame, the earlier editing request is followed by Stopped thinking. Below it is a new user message, Hello?. Only below that follow-up do the next captions appear [S11]:

```
earlier editing request
    Stopped thinking

new user message: Hello?
    [book-like icon]
Reviewed and clarified
```

```
Fishing the Stream

article edits and scope
    [book-like icon]
Preparing evidence package
```



The next supplied frame retains the review caption while the lower line reads Shaping the conclusion [S12]. A later frame

retains the review caption while the lower line reads Finalizing deliverables [S13]. The sequence is observational; it is not a timing measurement.

This matters because the first conversational interpretation was too strong: it described these as internal finishing stages continuing after thinking had stopped. That interpretation skipped Hello?.

The frames support something narrower: **after a user follow-up, activity referring to the earlier editing task became visible**. They do not tell us whether work restarted, reused existing files, resumed an internal checkpoint, or merely

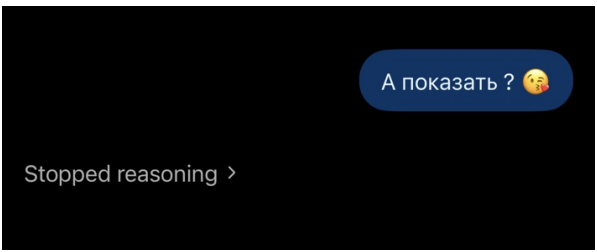
received updated UI captions. They also do not establish that the preceding attempt continued uninterrupted. The book-like icons do not identify a worker or prove that a particular file was read.

One more label: Stopped reasoning

A separate pair of captures shows a translation task with a Thinking label [S14]. In the next supplied frame, the user has added a short follow-up. A different label appears below that message [S15]:

```
Stopped reasoning
```

The composer still displays the blue Stop control.



This adds another label to the observed inventory. It does not establish an additional reasoning worker, an OOM, or the cause of the stop. The captions alone cannot distinguish user interruption, a failure, or another state transition. Likewise, the earlier English message is a statement of intent, not proof that the promised output was already complete.

The limit that improves the record

The useful addition is a sharper reading rule: **preserve message boundaries, not just the order of status strings**. A stopped label above a new message and activity below it are not sufficient evidence of a single surviving process.

Supporting crops S10-S15 are included in the evidence folder. The state labels and their ordering are the relevant observations.

10. Chat Titles Behave the Same Way

Chat titles also appear to be generated from early conversation context.

In one case, a title clearly reflected the opening topic but unexpectedly switched one word into another language.

Observed

- the title tracked the beginning of the conversation;
- it remained anchored there even after the conversation moved elsewhere;
- it exhibited language drift.

Working hypothesis

Chat titles are generated from early context and are not necessarily continuously recomputed.

The broader pattern is interesting:

- status labels summarize the current task;
- titles summarize the opening context;
- memory can summarize durable user information.

Different summaries.

Different lifetimes.

Different purposes.

11. Memory Is Not Passive

The memory system produced two especially interesting behaviours.

First, a visible **Memory updated** event appeared before the main response had finished.

That suggests memory writing may be a separate side effect with its own lifecycle.

A possible behavioural sequence is:

```
user message
-> memory write completes
-> main response still
generating
```

Again, that is a model of the behaviour, not a claim about implementation.

The second observation was more important.

Sometimes memory was updated even when the user had **not** explicitly said "remember this" or "save this."

Observed

The system can appear to decide autonomously that some information deserves persistence.

Working hypothesis

There is likely some policy that estimates long-term usefulness, stability, or future relevance.

That is understandable.

A memory feature would be weak if it required an explicit save command every time.

But it introduces a symmetrical risk:

1. the system can decide what deserves to become memory;
2. the system can later decide when that memory deserves to resurface.

That creates two failure modes:

- **unsolicited memory creation**

- **unsolicited memory resurfacing**

Which leads directly to the next observation.

12. A New Chat Is Not Always a Blank Page

After restarting the mobile app, I sometimes landed in a new chat rather than the previously active one.

On that apparently blank screen, a personalized prompt or suggestion was already present.

It referenced a real long-term project.

That feels helpful when the topic is neutral.

It could feel very different if the surfaced topic were painful, sensitive, or simply unwanted at that moment.

UX implication

A personalized cold start can become an unsolicited memory-surfacing mechanism.

> **New chat. Blank page. Except for the part where it already knows what you should be doing.**

The selection algorithm is unknown.

It may be based on recency, importance, goals, ranking, memory, or something else.

Calling it "random" would overstate what the evidence shows.

From the outside, it is simply unpredictable.

13. Image Routing Has Something That Feels Like Opcodes

After repeated image-generation work, I noticed a practical control surface.

Short follow-ups such as:

```
Draw.
Go.
Proceed.
```

often went straight into another image operation.

But:

```
Think, don't draw.
```

switched the interaction back into discussion in the observed exchanges. I did not establish a success rate or a universal guarantee.

These are not official commands.

They are field-tested behavioural controls.

Which makes them feel hilariously close to

image-pipeline assembly language.

Working hypothesis

After an image turn, a routing or optimization layer may classify short follow-ups as continuation commands and take a shorter path into image generation.

That would make engineering sense.

If the user asks to remove a tiny visual blemish, why spend a full conversational reasoning turn rediscovering the entire task?

A shortcut saves latency and compute.

The problem is not the optimization.

The problem begins when the shortcut preserves the wrong thing.

13.1 A Smaller Instruction

The useful phrase became:

```
Stop. Think. Don't draw.
```

The shortest part of that instruction was the most interesting:

```
Don't draw.
```

It restricts an action rather than specifying the contents of an image. It asks the system not to

draw, leaving room to discuss what to do next.

There is more than one way to implement that behaviour. A model may obey the instruction and refrain from calling a tool. A coordinator may change its choice of action. A larger workflow may change direction. The result alone does not distinguish them.

I kept the assembly-language analogy, but not as a claim that I had found internal opcodes. The useful fact was smaller: **a short instruction could change the kind of output I received, not merely its style.**

A second pattern: Are you there?

After some unfinished tasks, a tiny follow-up:

```
Are you there?
```

was followed by renewed activity or delivery of an output, without a full restatement of the original task.

That made it useful as a conversational request to try again. It did not establish a literal retry command or recovery from a saved execution checkpoint.

The later session failure matters here: both Are you there? and Hello failed before the client was restarted. A new Hello then

succeeded. This is not a controlled comparison between two words in the same system state, and it does not prove that "Are you there?" is semantically empty or forces a status check.

The practical observation survives the narrower interpretation: some short follow-ups were enough to continue the work; they were not reliable recovery commands.

13.2 The Aquarium Request That Produced Two More Portraits

Later, the same conversation supplied a particularly clear mistake.

After a portrait-generation exchange, I wrote:

```
Let's carefully add a couple more fish to the aquarium.
```

In the established vocabulary of this conversation, the "fish" were observations for these field notes and the "aquarium" was the collected material. I was asking to return to the notes, not to continue the portrait session.

The response was two more portraits.

Not fish. Not an aquarium. Not a question about which notes to add.

I then wrote:

```
Don't draw. We've drifted
again.
```

The next response was text. The assistant acknowledged that it had continued the image task instead of recognizing the metaphor. The visible conversation then returned to preparing a Markdown edit.

Evidence source: the message/output sequence in this conversation. S09 captures the later Markdown-readiness step; it does not itself capture the two unwanted generations.

This is a stronger observation than saying that a vague "go ahead" sometimes generates another picture: the output

continued an earlier visual activity despite the different intent of the current request.

Still, I cannot tell whether a separate router misclassified the message, a model chose the wrong tool, or prior visual context dominated its interpretation. The assistant's acknowledgement describes the mistake; it is not privileged telemetry about its cause.

What the record does show is:

> **An unintended image action was followed by a textual correction; the following reply returned to discussion instead of generating another image.**

The practical control worked in that exchange. Its internal wiring remains unknown.

14. Sticky Inference

This became the central pattern of the investigation.

A system makes a hidden inference:

```
this should be preserved
```

or:

```
this is important
```

or:

```
this turn is still active
```

Most of the time, that inference is useful.

But if it is wrong and persistent, the error becomes state.

That pattern appeared in multiple places:

- memory can preserve information the user did not explicitly ask to save;
- image editing can preserve a malformed feature across retries;
- UI state can preserve an "active" turn after the work is done;
- late events can preserve old task state into a newer turn.

I started calling this:

> **Sticky inference**

or:

> **Autonomous shortcut with persistence**

The shortest version is still the best:

> **A wrong guess becomes state.**

15. Generation and Filtering Can Disagree

Safety behaviour produced another architectural question.

Imagine a completely harmless request:

> *Draw a wholesome rescue scene where a child hugs a bear after danger has passed.*

The user intent is benign.

But suppose the generator adds an inappropriate visual detail that the user never requested.

A post-generation filter sees the actual candidate and correctly blocks it.

From the filter's point of view, everything worked.

From the user's point of view:

> *I asked for a hug. Why was I refused?*

This is an important UX distinction.

> **Post-generation filtering protects the output. It does not necessarily protect the UX.**

A robust system may evaluate both intent and candidate output, but I did not map the full safety path and do not know how many stages exist. What I observed was narrower: some blocks became visible only after generation had already begun.

> **The user asked for a hug. The generator improvised. The filter filed the complaint.**

16. Semantic Drift Before Filtering

A related problem appeared before any visible rejection. Across more than one image-generation system, ambiguous romantic prompts could drift toward more sexualized imagery than the wording required.

A phrase as simple as:

```
a man kisses a woman
```

can describe a cheek kiss, a brief affectionate kiss, an embrace, or a quiet romantic moment. Yet the generated composition may choose a more sexualized convention.

Observed

Ambiguous romantic language can be resolved more sexually than the wording requires.

Working hypothesis

The generator may have priors that over-associate some romantic actions with sexualized visual conventions. Possible contributors include captions, preference optimization, prompt rewriting, sampling, or safety tuning.

Output alone cannot tell us which mechanism is responsible or what training data produced the behaviour.

The practical lesson was more useful than the causal guess: clarify the intended scene **before** generation rather than trying to fight a filter after the generator has already drifted.

17. A Reproducible Bug in Model Behaviour

This was the most interesting finding.

Not because it produced the strangest image.

Because it was reproducible.

Traditional debugging asks:

Which line of code is wrong?

Model debugging sometimes asks:

Which part of the previous answer refused to die?

The starting point looked completely innocent:

a photorealistic reference image of a woman sitting on a motorcycle, with small instructional labels around the scene.

It looked almost like stock photography.

That image later became a reproducible test fixture.

E-01 — Motorcycle Reference Image

Why it matters

This image appears to create a strong **head/face identity anchor** while leaving the **neck-to-torso relationship underconstrained**.

In later generations, the body, clothing, and pose may be rebuilt substantially, while the head/neck configuration remains suspiciously persistent.

Reproduction hint

1. Use the motorcycle image as the primary identity reference.

2. Ask for a substantially different body pose and setting.
3. Observe whether the body is regenerated while the head/neck orientation remains partially inherited from the original reference.

Short caption

Reproducible evidence: strong head persistence across pose changes.

18. What Made the Bug Interesting

The failure was not simply "the model drew a bad neck."

The body could change.

The clothing could change.

The pose could change.

The scene could change.

Yet the head/neck relationship remained eerily similar to the original reference.

That suggests some form of partial reuse, anchoring, or region-level persistence.

Then came the critical test.

A new instruction explicitly required the system to:

```
fully recompute head, neck, shoulders, and upper torso anatomy
```

under the new pose.

The defect did not appear in that test.

That does not tell us whether the root cause lives in:

- the model weights;
- the image-edit pipeline;
- a reference-preservation mechanism;
- prompt transformation;
- latent reuse;
- or an interaction among several components.

So the careful statement is:

> **I found a reproducible defect in model behaviour. The**

exact layer responsible may be the model itself, the edit pipeline, or their interaction.

That is still a meaningful result.

It is not "an image came out weird."

It is a black-box behaviour with:

- a reproducible trigger;
- a repeated failure mode;
- and a mitigation.

And it fits the larger pattern:

The model remembered the wrong thing.

19. The Stream Finally Confessed

The earlier theory about stuck completion state had always been plausible.

But plausible is not enough.

Then the client displayed this:

```
Streaming interrupted.
Waiting for the complete
message...
```

That sentence is unusually revealing.

The client is explicitly telling us two things:

1. it knows streaming has been interrupted;

2. it is still waiting for the completion message.

That is an internally awkward state.

The live delivery path is known to be broken.

Yet the state machine remains waiting for a terminal event associated with that path.

I decided not to touch anything.

I waited.

The client remained stuck.

This mattered because earlier observations had already suggested that once the client entered this state, waiting alone did not repair it.

Then I restarted the app.

After restart, the final state became available.

What this supports

The strongest evidence-backed formulation I am comfortable with is:

> **The client can detect that streaming has been interrupted, remain stuck waiting for terminal completion, and recover the final state after the app is restarted.**

That is much stronger than saying:

> *"maybe the completion packet got lost."*

And it is more precise.

I do **not** know whether the exact same network message is replayed after restart.

Several recovery mechanisms are possible:

- reconnect and resubscribe;
- final-state re-fetch;
- replay;
- polling;
- reconciliation;
- retry against durable server-side state.

But behaviourally, one thing is clear:

> **the normal live-stream path fails to recover itself, while restarting the app does recover the completed state.**

That points strongly toward a client-side recovery/retry/reconciliation problem after stream interruption.

Why the interruption itself matters

This is also where the old WebSocket lesson comes back.

The interruption is not an exotic edge case.

Long-lived streaming connections fail.

The architecture must assume that.

If the client can detect interruption but still remain trapped in "waiting for complete message," then the problem is not merely transport reliability.

It is recovery logic.

The robust system is not the one whose stream never breaks.

It is the one whose state machine knows what to do when it does.

20. A Different Failure: Stopped analyzing

Then a failure appeared that did **not** look like the interrupted-stream case.

The task was unusually heavy: reconstruct a long Markdown document while preserving chronology and incorporating a large amount of earlier conversational context.

The run stalled.

The composer still showed **Stop** instead of the normal send control.

I restarted the application.

Unlike the interrupted-stream case, restart did **not** heal the state.

The same active-task condition came back.

Then I pressed **Stop** manually.

That released the composer. The stopped-analysis label captured in this episode was:

```
Stopped analyzing
```

I sent a follow-up:

```
Are you there?
```

and a new progression appeared:

```
Thinking
Analyzing
```

That sequence is important because it looks like a different failure category from transport interruption.

What is directly observable

- the interface displayed stopped-analysis wording;
- the composer still displayed **Stop**, giving the appearance of ongoing activity;
- application restart preserved that active state;
- explicit user cancellation changed the composer state;
- the next user message started a fresh Thinking -> Analyzing progression.

A useful behavioural model is therefore:

```
parent run: ACTIVE
    |
    +-- analysis phase:
STOPPED
```

rather than:

```
stream: INTERRUPTED
run: COMPLETED
client: failed to receive
completion
```

That distinction matters.

In the earlier streaming case, restarting the app recovered the final state.

Here, restart reproduced the stuck run.

Only explicit cancellation released it.

Why would analysis stop?

This is where the evidence ends. Worker failure, memory pressure, a deadline, preemption, a tool failure, cancellation, or orchestration trouble could all produce superficially similar symptoms.

The unusually large document task makes resource pressure an attractive hypothesis because it combines long context, extracted file content, intermediate structure, rewritten output, and tool state. But there are no

memory measurements or internal logs.

So the strongest statement I am willing to make is:

> **A stopped-analysis label can coexist with a still-visible Stop control. In this episode, restart did not clear the condition, while explicit Stop did. The child/parent interpretation remains a model, not an inspected state machine.**

And the most important unanswered question is still:

> **What stopped the analysis?**

A less technical summary is probably just as accurate:

> **The task took on more than it could carry.**

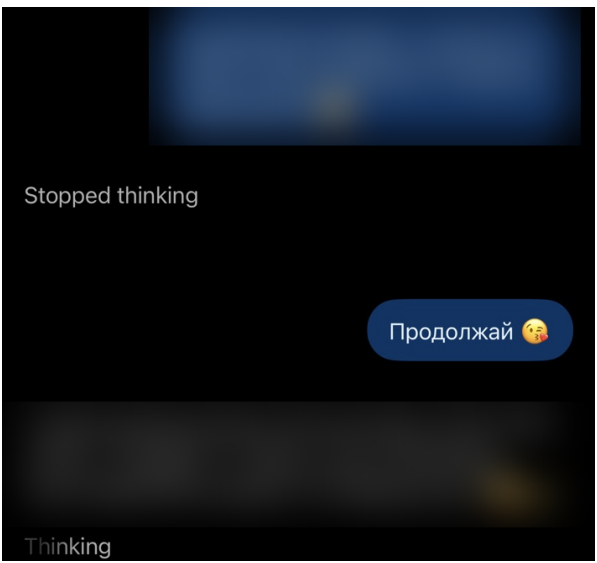
That line makes no architectural claim. It simply describes the observed outcome unusually well.

20.1 A Continuation That Shows Text Before Thinking

One screenshot adds a useful limitation to any literal reading of the labels. It shows the following top-to-bottom order [S08]:

```
request for a photograph
using an attached reference
Stopped thinking
user: Continue.
```

```
assistant: [visible prose]
Thinking
```



Visible assistant prose appears between the continuation request and the later Thinking label. The request and continuation were predominantly Russian.

The image does not show the final outcome of that attempt. It cannot tell us whether the prose is a plan, commentary before a tool call, or some other intermediate output. Nor does it tell us why the earlier activity was marked stopped.

It does show that **visible prose is not necessarily the final response**, and that a generic activity caption can be present below text that has already reached the user.

This also prevents a convenient shortcut in the analysis. Stopped thinking is not, on its own, proof that a large model process died. Continue. followed by activity is not proof that the same worker

resumed or that a checkpoint was restored. And English prose in a Russian exchange does not identify a new executor.

The screenshot records a continuation with intermediate output. It does not supply a process trace.

21. The Retry Button With No Visible Recovery

A third failure category appeared later the same day.

This time the problem happened **before** any visible reasoning phase.

The request contained text plus several media attachments, including a video.

Instead of entering the normal task lifecycle, the UI reported:

```
The request timed out.
```

and displayed a prominent orange:

```
Retry
```

I screen-recorded the interaction and pressed it.

The button visibly accepted the tap.

Nothing else visibly changed.

No new status.

No progress indicator.

No new attempt in the transcript.

No transition into thinking or analyzing.

The same timeout state remained on screen.

That does not prove that no retry request reached the backend.

But it does establish the behavioural observation:

> **Retry visibly accepted the tap, but produced no observable state transition.**

This failure is different from both earlier cases.

```
Streaming interrupted
    -> a run already
existed
```

```
    ->
delivery/reconciliation
failed

Stopped analyzing
    -> a child execution
phase stopped
    -> the parent run still
appeared active

Request timed out
    -> failure occurred at
or before
request/upload/submission
    -> a full run may never
have started
```

The other useful observation is that the **entire** user request failed

even though the visible problem appeared tied to an attachment.

The text remained visible.

Other media previews remained visible.

But the request as a whole did not proceed.

That suggests an atomic boundary somewhere around request submission or attachment preparation.

A plausible, but unproven, failure story is:

```
composer contains text +
media
-> one upload/request
component times out
-> submission does not
```

```
obtain a valid resumable
state
-> UI keeps local text and
thumbnails
-> Retry control remains
visible
-> there may be nothing
valid left for that control
to resume
```

Or the retry may have been sent and immediately failed again.

The recording cannot distinguish those cases.

What it does show is simpler and stronger:

> **The UI preserved the shell of a failed request after the request itself may no longer**

have had a recoverable execution state.

That gives us three different kinds of failure with three different recovery behaviours.

The stream could be reconciled by restart.

The stuck analysis required explicit cancellation.

The timed-out media request presented a Retry control that, at least visibly, went nowhere.

One interface.

Three failure boundaries.

Three recovery stories.

21.1 The Failed Run That Kept Its Work

A later failure made the Stopped analyzing case more interesting.

The task again involved reconstructing a substantial Markdown document from a long conversation history. I suspected high resource demand; I had no memory measurements.

The visible analysis failed.

After subsequent follow-ups, a finished Markdown file became available.

The artifact is stronger evidence than an assistant sentence saying that it "already existed." But its later availability alone does not

establish which attempt created it or exactly when the write completed.

> **A visible failure did not prevent eventual retrieval of the requested file. Survival of a prior write is one possible explanation, not a commit trace.**

A simple behavioural sequence is:

```
gather context
-> analyze / synthesize
-> create artifact
-> final response /
completion notification
```

The interesting possibility is that these phases are **checkpointed** or independently committed.

For example:

```
phase 1: gathered state
persisted
phase 2: synthesis state
persisted
phase 3: artifact persisted
phase 4: completion
delivery failed
```

That would explain two otherwise interesting observations:

1. a finished artifact was available after a failed-looking run;

2. subsequent manual retries sometimes felt faster than the original attempt.

The second point matters.

Repeated attempts sometimes felt faster, but I did not collect controlled timings. Latency alone cannot distinguish checkpoint reuse from shorter output, changed workload, caching, tool results already present in the conversation, or normal variation. It is a clue to record, not evidence of persisted synthesis state.

I considered a resource-pressure interpretation:

```
worker receives large
memory-intensive task
```

```
-> gathers context
-> builds intermediate
representation
-> begins synthesis
-> memory pressure rises
-> worker dies or is
terminated
```

On another attempt, the worker may get farther:

```
reuse persisted work
-> create artifact
-> fail while finalizing /
reporting completion
```

This is consistent with an out-of-memory or worker-crash hypothesis, but it does not prove one.

The stronger black-box claim is:

> **Preserved intermediate work is a candidate explanation. Which results, if any, survived each attempt is not established here.**

I avoid calling these phases "transactions" in the strict database sense.

A safer term is:

> **checkpointed or phase-committed work.**

Why was no successful automatic recovery visible?

I did not observe a successful automatic recovery in these episodes. That does not prove there were no internal retries.

If intermediate work can survive, a conservative orchestrator still has reasons not to replay blindly: the failure may be deterministic, the operation may already have side effects, the phase may not be idempotent, or another attempt may simply repeat an expensive failure.

A plausible policy would therefore be:

```
fail
-> keep whatever was
already committed
-> report the failure
-> wait for explicit user
action
```

Under that hypothesis, a tiny follow-up such as Are you there?

can be useful without being a literal resume opcode. A later run may reuse existing conversational or artifact state, or it may simply start fresh with enough context to continue.

21.2 The Conversation Itself Can Roll Back

Then a more serious failure appeared.

The sequence was:

1. a normal user message was sent;
2. the client displayed Hmm... Something seems to have gone wrong.;

3. a follow-up Are you there? produced the same fallback;
4. a neutral Hello also produced the same fallback;
5. several recent user messages disappeared from the visible chat;
6. the application was restarted;
7. a new neutral Hello succeeded;
8. I experienced the later answer as returning to an older topic rather than addressing the new screenshot as intended.

This is materially different from a worker failure.

The visible transcript itself had changed after the fact.

The strongest observation is:

> **I observed previously visible turns disappear and, after recovery, received an answer that seemed anchored in older context. The first is a reported transcript change; the second is an interpretation of answer relevance.**

That implies a dangerous distinction:

```
visible transcript
    !=
durable conversation state
```

One possible model is optimistic UI:

```
user sends turn
-> client immediately
renders bubble
-> backend commit happens
later
```

If the execution/session path fails before durable commit:

```
client shows turn
-> commit fails
-> restart reloads last
durable checkpoint
-> uncommitted turn
disappears
```

But several other explanations remain possible:

- stale replica;
- failed message commit;
- read-after-write lag;

- client/server cache divergence;
- attachment linkage failure;
- context reconstruction from an older snapshot.

So I will not claim a particular storage architecture.

The evidence supports a narrower and more important statement:

> **Visibility in the chat was not a sufficient preservation check in this episode. These observations do not locate the server-side commit boundary or prove permanent deletion.**

That means the chat transcript cannot automatically be treated as an append-only research log.

For this investigation, screenshots, screen recordings, and exported artifacts are not merely illustrations.

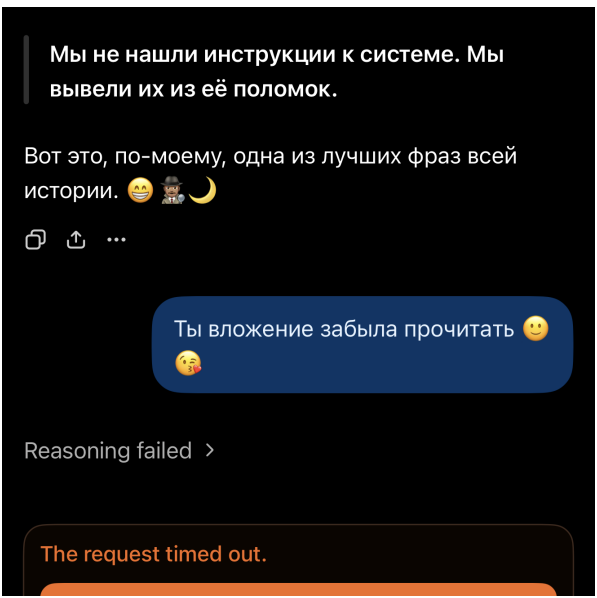
They are independent evidence.

21.3 The Turn Survived, the Failure Card Did Not

A later pair of screenshots made the persistence boundary more concrete.

In the first capture, the same user message was still visible while the UI showed:

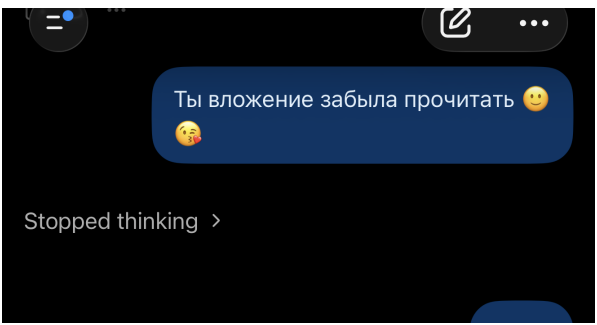
```
Reasoning failed
The request timed out.
Retry
```



In the later capture, the user message was still present, but the timeout card and Retry button were gone. The visible failure state had changed to:

Stopped thinking

A new Are you there? turn and a normal reply were now below it [S17].



The important observation is not which backend component failed. It is that **the message bubble and the failure presentation had different visible lifetimes**.

> **The turn persisted. The failure state did not.**

This does not prove that the timeout card existed only in client memory, that a server record was deleted, or that Stopped thinking replaced the same backend state. It only shows that the visible error presentation was not preserved as an append-only part of the transcript.

That distinction matters because the UI can later look calmer than the event that actually occurred.

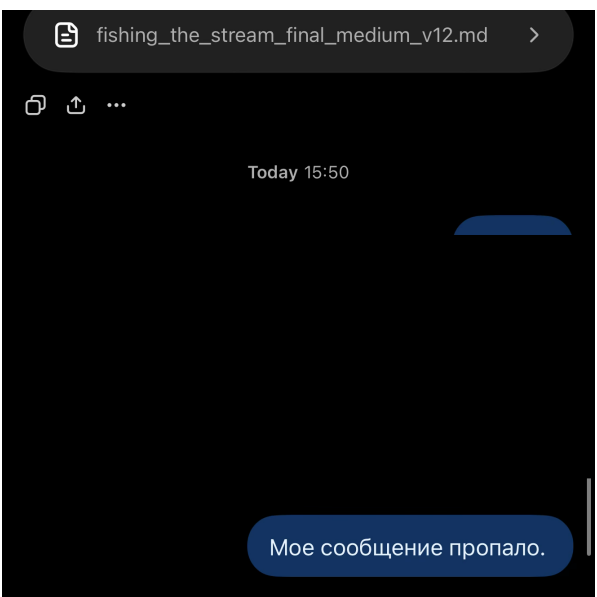
21.4 A Failed Branch Can Disappear

A separate episode went one step further.

The observed sequence was:

1. a user turn with several attachments was visibly rendered;
2. the request failed;
3. a Retry control appeared;
4. tapping Retry produced no observable transition;
5. the composer remained usable;
6. a new neutral Hello was submitted successfully;
7. the previously visible failed user turn, its attachments, and its failure / Retry presentation were no longer present in the visible transcript.

A later screenshot shows the recovered conversation beginning with the successful Hello, followed by my explicit note that the previous message had disappeared [S18].



The after-state screenshot cannot, by itself, reconstruct the missing

branch. The sequence above comes from my direct observation of the transition. So the strongest statement is deliberately narrow:

> **The failed turn and its error state were both removed from the visible conversation after the next successful submission.**

This is stronger than merely saying that a Retry action failed. In the visible UI, the whole failed branch ceased to exist.

It still does not establish permanent server-side deletion. The omitted branch could reflect failed commit, reconciliation to an older durable state,

stale-replica recovery, attachment-link failure, or another persistence boundary.

For black-box use, however, the practical rule is simple:

> **Visible is not durable. Retry is not recovery.**

And a second rule follows:

> **A transcript that has reconciled after failure may conceal the fact that a failure ever happened.**

Screenshots matter here because they preserve an event that the later UI may no longer show.

21.5 A Late Catch: The Message Vanished, the File Survived

This one arrived after the manuscript was already finished.

On 24 September 2026 I returned to a conversation in which I had uploaded a five-minute screen recording the day before for an unrelated analysis. The attachment was no longer present in the visible message history. I noticed the absence because the recording itself was the object I wanted to discuss next.

The interesting part came after the disappearance. A separate file-retrieval surface available to

the assistant still listed the original artifact by its original filename:

```
ScreenRecording_09-23-2026 12-15-47_1.mp4
```

The file was 213,074,795 bytes. It could be retrieved again and opened as a valid MP4, with a duration of about 302.77 seconds [E20].

That gives a clean inversion of the earlier artifact-lifetime observation in Section 2.

There, conversation state could retain knowledge of an artifact after direct access to its bytes was gone.

Here, the visible conversation lost the attachment while the artifact bytes remained retrievable elsewhere.

Observed

- the user-facing transcript no longer showed the uploaded video;
- a separate file-retrieval surface still exposed the original artifact;
- the bytes could be retrieved and decoded successfully;
- therefore absence from the transcript did not imply deletion of the artifact.

Not established

This does **not** establish why the attachment disappeared from the transcript, whether a message record was deleted, hidden, rolled back, or merely no longer rendered, whether the surviving file belongs to the same persistence system as the conversation, how long it will remain retrievable, or whether the ordinary client UI exposes any independent recovery path.

So the narrow conclusion is:

> **Visible transcript presence and artifact availability can diverge in both directions.**

And the practical rule is even simpler:

> **Missing from the conversation is not the same as missing from storage.**

Field-note nickname:

> **The berry does not exist.**

22. Sidebar: A Second Predictive System Failed Differently

While preparing evidence for the image-generation bug, I noticed an unrelated prediction from the iOS keyboard:

```
7 + 6 = 594
```

The arithmetic looked absurd until the nearby numbers revealed a coherent interpretation:

```
6 × 14 × 7 + 6 = 594
```

The calculation is correct. The inferred task is not.

That makes this a useful side example of the same broader failure pattern: a predictive system can remain locally coherent while choosing the wrong context.

> **The arithmetic was correct. The reasoning was locally consistent. The answer was absurd.**

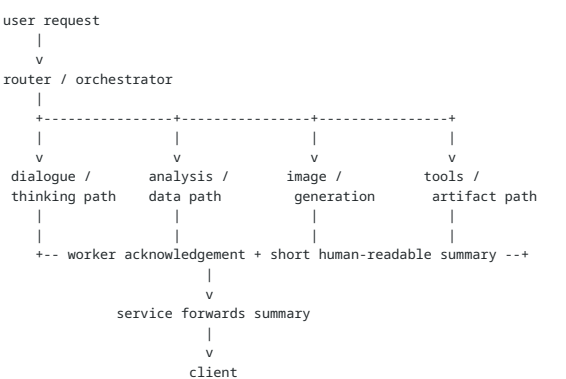
The image system preserved the wrong visual context; the keyboard selected the wrong numeric context. In both cases, extra context did not rescue intent. It displaced it.

Over-contextualization can be just as dangerous as under-contextualization.

The coincidence was fitting: evidence gathered for one model-behaviour bug produced a second example from another predictive system. The investigation had started generating its own side quests.

23. A Possible Black-Box Model

The following is one candidate architecture, not a reconstruction established by the screenshots. The boxes organize a worker/acknowledgement/observer hypothesis; the new evening evidence does not identify those boxes.



A worker may itself decompose work further:

```
worker
  |
  +-> subtask A -> summary
A
  +-> subtask B -> summary
B
  +-> subtask C -> summary
C
  |
  +-> aggregate / continue
```

That could explain why a visible status can change language, or even contain mixed-language fragments, without the user's own language behaviour accounting for it.

Again, that is only a working hypothesis.

A simpler multilingual-generation bug remains possible.

The failure path may be different:

```
worker
  |
  X crash / stop / lost
lease / abnormal
termination
  |
health observer /
supervisor
  |
machine outcome
  |
service maps outcome to
fixed English description
```

```
  |
client
```

This would fit the visible contrast between contextual progress narration, translated where needed:

```
Analyzing statuses
Classifying run states
Reviewing diagram visuals
```

and dry failure descriptions:

```
Stopped thinking
Stopped analyzing
Error analyzing
The request timed out.
```

The system may also preserve work by phase:

```
gather
  -> checkpoint

analyze
  -> checkpoint

create artifact
  -> checkpoint

finalize / deliver
```

A failure late in the chain would then leave earlier work available to the next run.

Finally, conversation persistence appears to have its own boundary:

```
visible client turn
    |
    v
```

```
durable conversation commit
    |
    v
context available to next
run
```

The rollback incident suggests those stages are not guaranteed to be simultaneous.

This entire diagram is a reasoning aid.

The real implementation may use queues, RPC, actors, workflow engines, shared workers, specialized models, or something else entirely.

An important competing explanation uses fewer inferred components: one conversational

model can choose tools while a client displays tool captions, source metadata, and generated activity summaries. It can also choose the wrong tool or misunderstand a metaphor. These screenshots do not rule that out.

The new evidence therefore helps distinguish presentation roles and user-visible actions. It does not select a worker topology, prove a queue, prove that summaries are forwarded verbatim, or establish automatic phase checkpoints.

24. What We Actually Know

The observations cluster more cleanly by boundary than by chronology.

Artifacts and context

- Local-looking /mnt/data/... paths can appear in the UI, and textual knowledge about an artifact can outlive direct access to its bytes.
- Conversation-visible references and temporary file access can have different lifetimes; the observed behaviour also suggests that active context is

not identical to full chat history.

- Rich pasted content can become a .webarchive-like artifact and pass through visible file-ingestion steps before use.

Status and presentation

- Activity labels can be task-shaped, rephrased, and multilingual.
- The same status area can mix retained history rows, changing narration, query text, counts, and icons.
- Stopped thinking, Stopped analyzing, Stopped reasoning,

Error analyzing, and timeout messages are distinct visible labels. They are not, by themselves, identified worker states.

- A visible Stop control can coexist with a stopped label, and an error card can disappear while the user turn remains.

Routing and tool choice

- Short post-image follow-ups can continue image generation even when the intended topic has changed.
- An explicit Don't draw correction returned the

observed exchange to discussion.

- The UI reported repository cloning alongside web search, which is consistent with task-appropriate tool selection but does not verify the underlying operation.

Generation behaviour

- A completed image can later be accompanied by an image-generation error.
- An image reference produced a reproducible head/neck persistence defect across pose changes, and an explicit recomputation instruction mitigated it.

- Generation can drift semantically before a safety rejection becomes visible.

Failure and recovery

- The client can report an interrupted stream, remain stuck waiting for completion, and recover the final state after the app is restarted.
- A Stopped analyzing episode survived restart until explicit Stop, after which a new turn produced fresh activity.
- A media-bearing request can time out while leaving its visible shell and a Retry control that produces no observable recovery.

- A requested artifact can become available after failed-looking attempts, although the record does not establish which attempt created it or whether any internal checkpoint was reused.

Conversation persistence

- Previously visible turns can disappear around failure/recovery.
- In one captured sequence, the user turn survived while its timeout/Retry presentation disappeared.
- In another observed sequence, a failed turn with attachments

and its error state disappeared after the next successful submission.

- In a later case, an uploaded video disappeared from the visible transcript while the original MP4 remained retrievable through a separate file-retrieval surface.
- The product eventually displayed an explicit conversation-level refusal: This conversation is too long to continue. Start a new chat and try again.

Operating the black box

- Small natural-language constraints can change the

kind of action produced, not just the wording of an answer.

- Staged document instructions requesting smaller units of work and explicit checkpoints produced intermediate files and a final assembly.
- Instability appeared to increase as the conversation became larger and more multimodal, but no controlled failure-rate-versus-size measurement was collected.

25. What We Do Not Know

The screenshots do **not** reveal the internal architecture. In particular, I cannot determine:

Runtime topology

Whether the system uses one runtime or many, whether workers are specialized, whether subtasks become subworkers, what language any service uses, or whether the visible task classes correspond to distinct models, configurations, or only presentation states.

Context and persistence

How active context is assembled; whether retrieval is vector-based; where durable conversation

commit occurs; which intermediate phases, if any, are persisted; or whether disappearing turns reflect failed commit, stale replicas, cache divergence, snapshot lag, attachment-link failure, or something else.

Files and tools

What /mnt/data physically maps to, whether temporary artifacts have a fixed TTL, whether the late-surviving upload reflects an independent artifact store, a library copy, or another retention layer, whether the reported repository clone succeeded, what a displayed website count actually counts, or whether tool

operations were sequential or concurrent.

Status and recovery

Whether progress summaries come from workers, a coordinator, or the client; whether failures come from a separate observer; why Stopped analyzing occurred; whether reconnect performs replay, re-fetch, polling, or reconciliation; and what Retry actually attempted in the timeout cases.

Image generation and safety

The exact safety stages, the source of semantic drift, the layer

responsible for the motorcycle-reference defect, or whether image continuation reflects a separate router, ordinary instruction following, tool choice, or some combination.

Conversation-size pressure

What quantity triggered the final conversation is too long refusal; whether that guard is near the true reliability boundary; and whether earlier failures were caused by conversation size, correlated with it, or emerged only under particular workloads.

Those unknowns are not weaknesses. They are the boundary of the experiment.

A black-box investigation becomes useful when it is disciplined enough to stop at that boundary.

26. The Field Rule

I started this investigation by following small anomalies.

A path.

A stuck Stop button.

A status label.

A memory event.

A stream that admitted it had been interrupted but kept waiting anyway.

A stopped-analysis caption beside a Stop control that remained on screen.

A Retry button with no observable recovery.

A generated image that remembered the wrong neck.

Then an arithmetic completion that was technically right for a question nobody asked.

None of those clues mattered much alone.

Together, they suggested a pattern:

> **The optimization is not the problem. The problem begins when an inferred shortcut becomes persistent.**

Or even shorter:

A wrong guess becomes state.

That became a useful editing rule for this collection of observations.

Because in modern AI systems, the most interesting bugs may not live in a line of code.

They may live in the moment when a system forms a reasonable interpretation, preserves it, and continues being reasonable long after reality has moved on.

26.1 The Conversation Was the Stress Test

Only near the end did the shape of the experiment become obvious.

No benchmark had been designed. One conversation simply accumulated real work:

```
text
+ images and video
+ uploads and generated
media
+ retrieval and tool use
+ artifact creation
+ retries and failed turns
+ long-lived conversational
state
```

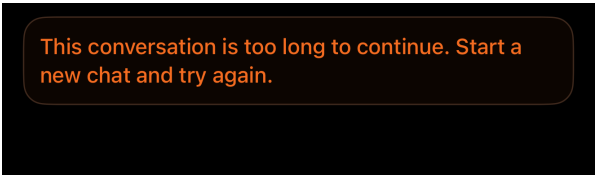
As that state grew, the conversation accumulated a wider variety of visible failures: stale activity, interrupted

delivery, stopped phases, timeouts, ineffective Retry controls, disappearing failure state, and a disappearing failed branch.

This is **not** a controlled reliability curve. I did not measure failure rate against token count, attachment bytes, or tool calls. The observation is only that instability became more visible as the conversation became larger and more multimodal.

Then the product exposed an explicit boundary [S19]:

```
This conversation is too
long to continue.
Start a new chat and try
again.
```



The message does not reveal what quantity reached its limit, and it does not prove that approaching that limit caused the earlier failures. The guard may simply be a product-defined limit and need not coincide with the point where reliability begins to degrade.

That is where the stack-overflow analogy is useful, provided it stays an analogy:

```
comfortable region
    -> increasing edge
behaviour
        -> uncertain margin
```

```
            -> explicit
hard stop
```

The conversation also exercised different visible capabilities: dialogue, file ingestion, image generation, retrieval, artifact creation, recovery, and persistence. That does not prove separate backend services. It does explain why one long-lived chat could expose several different failure boundaries.

> **The conversation was not designed as a stress test. It became one by accumulation.**

And the failures changed how I operated it. That is the final part of the story.

27. The Catch Hiding in Plain Sight

By the end, I was no longer only describing the answer I wanted. I was also constraining **how the next piece of work should happen**.

```
Don't draw.
    -> return to discussion
instead of another image

Are you there? / Continue.
    -> request continuation
without restating the whole
task

Work in small stages. Save
```

```
a checkpoint after each
stage.
    -> reduce work-unit
size and materialize
intermediate artifacts
```

These are not documented opcodes. Are you there? was not a reliable recovery command, and Don't draw is not an infallible switch. The assembly-language analogy is useful only at the user-facing level: a short phrase can constrain the next action, not merely the style of the answer.

I call that **routing engineering**. It means influencing whether the system discusses, invokes a tool, continues earlier work, or makes an intermediate result explicit. Ordinary instruction following

could explain it; so could a separate router; so could both.

> **In that user-facing sense, the conversation box is also a natural-language control surface.**

The chronology is what makes this interesting. The control phrases appeared **after** failures revealed weak boundaries. The checkpoint workflow was not a stylistic preference; it was an operational workaround derived from repeated document failures.

The same adaptation appeared in image work. When an affectionate prompt drifted toward a more sexualized scene, the useful move was not to defeat

a filter. It was to specify the intended non-sexual scene more clearly before generation. Over time, the conversational layer itself could translate shared intent into a more precise generation request.

That yields the most useful loop in the whole investigation:

> **Observation -> architectural hypothesis -> operating instruction -> another observable result.**

Or, more simply:

> **I did not find the system manual. I derived parts of it from the failures.**

That still does not reveal the hidden architecture. It reveals something more practical: a black box can teach you how to operate it without ever surrendering its internals.

Useful control. Imperfect interpretation. Unknown implementation.

That is enough.

Do not be afraid to accidentally disturb a system's familiar behaviour.

Normal operation tends to hide structure behind routine. An unexpected failure, a broken assumption, or an unusual state can expose boundaries and

relationships that the happy path keeps invisible.

> **A failure is not a catastrophe. It is an invitation to investigate.**

Sometimes it is worth asking:

> **What did the system just reveal about itself?**

There may be a large fish in there.

> **The fish are caught. The investigation is closed. Serve after removing the bones.**

Appendix: Field Notes From the Museum of Expired Food

A few lines survived the investigation and deserve to remain attached to the evidence:

The result was obtained. The method is not reproducible.

Work completed. Waiting for completion continues.

The work is finished. The indicator was not informed.

The image was saved. Image generation failed.

The path is known. The whereabouts are not.

The user asked for a hug. The generator improvised. The filter filed the complaint.

Expired does not mean useless. Some content may still be editable.

Retry was available. Recovery was not observable.

The turn persisted. The failure state did not.

Visible is not durable. Retry is not recovery.

The message vanished. The file survived.

> **I did not find the system manual. I derived parts of it from the failures.**

And one methodological rule:

> **Do not confuse a good story about the architecture with evidence of the architecture.**

That rule has already saved this investigation from several very attractive mistakes.

Evidence Appendix: Selected Captures and Field Notes

Most entries below are cropped excerpts of supplied screenshots, not recreated UI. The order follows the supplied sequence,

not a recovered event log. Later captures document the timeout-state disappearance, the post-loss transcript, and the explicit conversation-length refusal. E20 is a compact field record of the late attachment-survival observation; it records only the facts checked in that session.

ID	Evidence
S01	Retained search/clone captions plus domain-specific search

S02

Same retained rows plus generic search caption

S03

Same retained rows plus finalization summary

S04

Website-count caption

S05

Search query presentation

S06

Task summary without a visible leading icon

S07

The same wording, now accompanied by a green icon

S08

Stopped label, continuation, visible prose, and further activity label

S09

Readiness question followed by a matching activity caption

S10

Editing/package captions

S11

Stopped label, new follow-up, then renewed activity

S12

Follow-up sequence with later shaping caption

S13

Follow-up sequence with later finalization caption

S14

Generic Thinking state from the translation task

S15

Short follow-up followed by Stopped reasoning

S16

User turn plus Reasoning failed, timeout card, and Retry

S17

Same user turn remains; timeout / Retry card is gone; Stopped thinking is visible

S18

Post-recovery transcript after a failed branch disappeared

S19

Explicit conversation-level refusal: chat is too long to continue

E20

24 Sep late catch: attachment absent from visible transcript while original MP4 remained retrievable

The included evidence manifest records filenames and SHA-256 checksums. The aquarium mistake is documented by the conversation's request and generated outputs; the evening screenshots alone do not show that full sequence.

These additions do not supply backend logs, memory measurements, worker identities, or a controlled experiment. Their contribution is a clearer account of what was visible, what the user requested, and what action followed.